# AI - Lab

## 3. CNNs in PyTorch



Hochschule Karlsruhe
Technik und Wirtschaft
UNIVERSITY OF APPLIED SCIENCES

Dr. Fabian Sigloch

WS 26/27

# Convolutional Neural Networks (CNNs)

Hochschule Karlsruhe
University of
Applied Sciences
HKA

- CNNs are deep neural networks that use **convolutional layers**.
- Convolutional layers exploit hierarchical structure: simple local patterns are assembled into increasingly complex patterns.
- CNNs are especially effective at recognising patterns and structures in images.

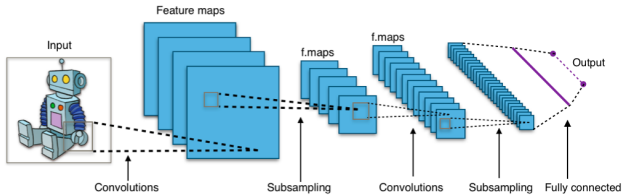



Image: https://de.wikipedia.org/wiki/Convolutional_Neural_Network

# Convolutional Layer

Hochschule Karlsruhe University of Applied Sciences HKA

- A convolutional layer slides a small learnable filter over an input image or feature map.
- At each location, it calculates the dot product between the filter and the local input region.
- The result is an **activation map**: high values indicate that the learned feature was detected.

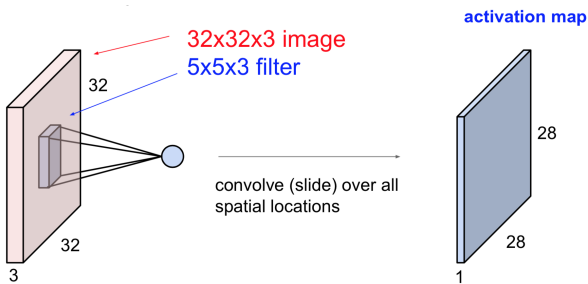



Image: CS231n

# Pooling Layer

Hochschule Karlsruhe
University of
Applied Sciences
HKA

- Pooling reduces the spatial dimensions of a feature map.
- Like convolution, it slides a window over the feature map, commonly with a stride greater than one.
- Max pooling outputs the largest value in each window; average pooling is another option.
- Downsampling reduces computation and makes features less sensitive to small translations.

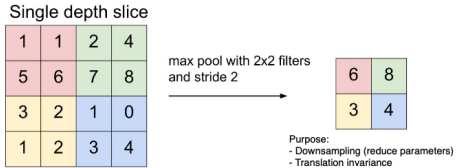



Image: CS231n

# CNNs



- CNN fundamentals were covered in the INFM110ML lecture; more depth will follow in the AI lecture.
- For a short recap, read this CNN blog post.

# CNNs in PyTorch



- PyTorch provides standard modules for the layers commonly used in CNNs:
  1. convolution: `nn.Conv2d`
  2. max pooling: `nn.MaxPool2d`
  3. fully connected layers: `nn.Linear`
  4. multi-class loss: `nn.CrossEntropyLoss`

# Example CNN in PyTorch

Hochschule Karlsruhe
University of
Applied Sciences
HKA

```python
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)
```

**CIFAR-10 input**

One image has shape `(3, 32, 32)`: three colour channels and $32 \times 32$ pixels.

- Two convolution + ReLU + max-pooling stages learn feature maps.
- The feature maps are flattened before three fully connected layers.
- The network returns class scores (logits), not softmax probabilities.

# `nn.Conv2d`: essential parameters

Hochschule Karlsruhe
University of
Applied Sciences
HKA

```
self.conv1 = nn.Conv2d(3, 6, 5)
```

**3**
input channels
(RGB)

**6**
output channels / filters

**5**
square kernel size ($5 \times 5$)

Other useful arguments include `stride`, `padding`, and `dilation`.

# Example CNN in PyTorch

Hochschule Karlsruhe University of Applied Sciences HKA

**One filter produces one activation map; several filters produce several maps**

```python
import torch.nn as nn
import torch.nn.functional as F


class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x


net = Net()
```

A convolution filter spans all input channels. `nn.Conv2d(3, 2, 5)` therefore creates two output feature maps from an RGB image.

# From feature maps to fully connected layers

Hochschule Karlsruhe University of Applied Sciences HKA

```
self.fc1 = nn.Linear(16 * 5 * 5, 120)

# after the second pooling layer:
x = torch.flatten(x, 1)
```

The first fully connected layer receives all values from the final convolutional output:

$$\text{input features} = C \times H \times W$$

- $C$: number of output channels
- $H$: feature-map height
- $W$: feature-map width
- Flattening changes `(N, C, H, W)` to `(N, C*H*W)`.

# CNNs in PyTorch

Hochschule Karlsruhe
University of
Applied Sciences
HKA

- The final layer returns raw class scores, called **logits**; there is no softmax layer in `forward()`.
- Why can we use cross entropy loss without explicitly converting logits to probabilities?

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(),
                      lr=0.001, momentum=0.9)
```

```
def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = torch.flatten(x, 1)
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    return self.fc3(x) # logits
```

# `CrossEntropyLoss` expects logits

Hochschule Karlsruhe University of Applied Sciences HKA

## What it does

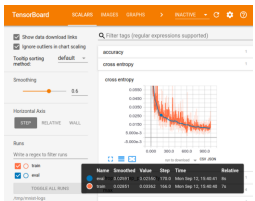
`nn.CrossEntropyLoss` combines a numerically stable `log_softmax` operation with negative log-likelihood loss.

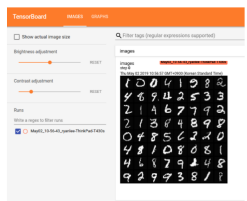
raw logits $\longrightarrow$ `CrossEntropyLoss` $\longrightarrow$ loss

- Do **not** add `softmax` before `CrossEntropyLoss`.
- Use `softmax` only when probabilities are needed for display or downstream decisions.

# TensorBoard






- TensorBoard is a user interface commonly used to visualise metrics and model artefacts in deep-learning projects.
- It originated with TensorFlow and integrates well with PyTorch.

# Assignment 3

Hochschule Karlsruhe
University of
Applied Sciences
HKA

**Preparation:**

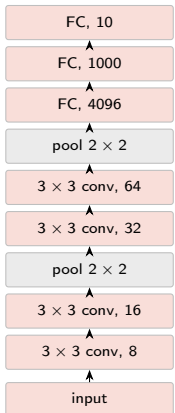
1. Work through the PyTorch CNN tutorial.
2. Work through the PyTorch TensorBoard tutorial and the instructions for TensorBoard in notebooks.

# Assignment 3

Hochschule Karlsruhe University of Applied Sciences HKA

**Task:**

1. Train the same network as in the PyTorch CNN tutorial.
2. Change the architecture and train it:
   - 2.1 `Conv2d`: $3 \times 3$ kernel, depth 8, ReLU
   - 2.2 `Conv2d`: $3 \times 3$ kernel, depth 16, ReLU
   - 2.3 max pooling: $2 \times 2$
   - 2.4 `Conv2d`: $3 \times 3$ kernel, depth 32, ReLU
   - 2.5 `Conv2d`: $3 \times 3$ kernel, depth 64, ReLU
   - 2.6 max pooling: $2 \times 2$
   - 2.7 fully connected: 4096, ReLU; then 1000, ReLU; then 10 logits



# Assignment 3

Hochschule Karlsruhe University of Applied Sciences HKA

**Task:**

3. Train on GPU and compare training time with CPU.
4. Log training loss and the network graph in TensorBoard.
5. Change the test metric: a prediction is correct when the true label is among the model's top three outputs. Print top-3 test accuracy.
6. Randomly select five top-3 errors from the test set. Log the images, true labels, and predictions in TensorBoard.
7. Show the TensorBoard widget at the end of the notebook.

# Assignment 3

Hochschule Karlsruhe
University of
Applied Sciences
HKA

**Bonus:**

- Try a deeper network or a different architecture to improve results.
- Explore `nn.Sequential` and use it to define a sequence of layers in the constructor.

# Submission Guidelines

Hochschule Karlsruhe University of Applied Sciences HKA

- Send one notebook file (`.ipynb`) by e-mail, including team name and team members.
- Use the subject: `[KI-Lab] Assignment 3 - <teamname>`
- Deadline: next Thursday, 23:59.

# Final words

Hochschule Karlsruhe
University of
Applied Sciences
HKA

- You can start working on the assignment now.
- You and your partner can organise your time as you see fit.
- I will be available for questions until the end of the slot at 1 p.m.
- Questions can also be asked in the chat; please do not post solutions.

- Any more questions?
- Happy coding!